Case Study 42: Student Marks Matrix Analysis

1. Problem Statement

Store student marks across multiple subjects in a 2D matrix (nested lists), calculate row-wise totals and averages for each student, calculate column-wise statistics for each subject, identify top-performing students, and flag subjects with low average performance.

2. Objectives

- Represent academic assessment data using 2D nested lists.
- Perform row-wise aggregations (student total/average) and column-wise aggregations (subject mean).
- Determine class rank toppers and flag subjects falling below a minimum class average benchmark.

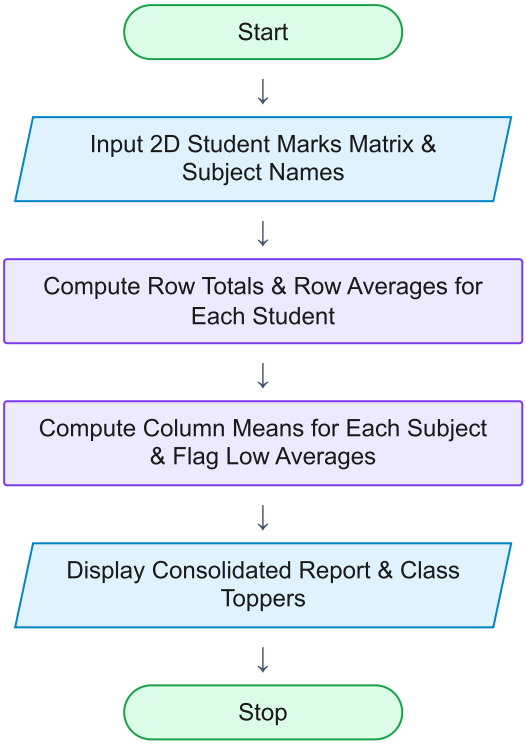
3. Assumptions

- Maximum score per subject is **100 marks**.
- Subject low-performance threshold benchmark is **60.0 marks**.
- Matrix dimensions: `N` students (rows) × `M` subjects (columns).

4. Algorithm

1. Start.
2. Initialize subject list and student marks matrix (nested list).
3. **Row-wise Analysis:**
   - For each student row, compute `total = sum(row)` and `average = total / num_subjects`.
   - Track the student with the highest total score (top performer).
4. **Column-wise Analysis:**
   - For each subject column `j`, calculate `col_sum = sum(matrix[i][j])` across all students `i`.
   - Compute `subject_avg = col_sum / num_students`.
   - If `subject_avg < 60.0`, flag subject as *Low Average Performance*.
5. Display itemized score card, subject statistics, and class toppers.
6. Stop.

5. Flowchart



6. Pseudocode

```
START INPUT subjects INPUT student_names, matrix num_students = length(student_names) num_subjects = length(subjects) max_total = -1 topper = "" // Row-wise Processing FOR i FROM 0 TO num_students - 1 DO row = matrix[i] total = SUM(row) avg = total / num_subjects IF total > max_total THEN max_total = total topper = student_names[i] ENDIF PRINT student_names[i], row, total, avg ENDFOR // Column-wise Processing FOR j FROM 0 TO num_subjects - 1 DO col_sum = 0 FOR i FROM 0 TO num_students - 1 DO col_sum = col_sum + matrix[i][j] ENDFOR subj_avg = col_sum / num_students IF subj_avg < 60.0 THEN status = "LOW PERFORMANCE" ELSE status = "SATISFACTORY" ENDIF PRINT subjects[j], subj_avg, status ENDFOR STOP
```

7. Complete Python Source Code

```
      # Student Marks Matrix Analysis def analyze_marks_matrix(subjects, students, marks_matrix): num_students = len(students) num_subjects = len(subjects) print("=" * 75) print(" ACADEMIC PERFORMANCE MATRIX REPORT") print("=" * 75) # Header Row header = f"{'Student Name':<16}" + "".join([f"{s:<10}" for s in subjects]) + f"{'Total':<10}{'Avg':<10}" print(header) print("-" * 75) top_student = "" top_score = -1 # Row-Wise Analysis for i in range(num_students): row = marks_matrix[i] total = sum(row) avg = total / num_subjects if total > top_score: top_score = total top_student = students[i] row_str = f"{students[i]:<16}" + "".join([f"{m:<10}" for m in row]) + f"{total:<10}{avg:<10.2f}" print(row_str) print("-" * 75) # Column-Wise Analysis print("\nSUBJECT-WISE COLUMN STATISTICS:") print("-" * 45) print(f"{'Subject':<16}{'Average Marks':<16}{'Status':<15}") print("-" * 45) low_perf_subjects = [] for j in range(num_subjects): col_sum = sum(marks_matrix[i][j] for i in range(num_students)) subj_avg = col_sum / num_students if subj_avg < 60.0: status = "NEEDS ATTENTION" low_perf_subjects.append(subjects[j]) else: status = "GOOD" print(f"{subjects[j]:<16}{subj_avg:<16.2f}{status:<15}") print("-" * 45) print(f"\nClass Topper : {top_student} (Total: {top_score}/{num_subjects * 100})") print(f"Low Avg Subjects : {', '.join(low_perf_subjects) if low_perf_subjects else 'None'}") print("=" * 75) # Test Data subject_list = ["Maths", "Physics", "Python", "English"] student_list = ["Ananya Rao", "Bikram Chand", "Chetna Jain", "Dinesh Kumar"] # Nested List Matrix (4 Students x 4 Subjects) matrix_data = [ [85, 78, 92, 88], # Ananya [55, 48, 62, 58], # Bikram [90, 85, 95, 91], # Chetna [40, 52, 58, 65] # Dinesh ] analyze_marks_matrix(subject_list, student_list, matrix_data)
```

8. Test Case and Expected Output

Evaluation Output for 4x4 Marks Matrix:

```
      ===========================================================================
ACADEMIC PERFORMANCE MATRIX REPORT
=========================================================================== Student Name Maths Physics Python English Total Avg ------------------------------------------------------------------------ Ananya Rao 85 78 92 88 343 85.75 Bikram Chand 55 48 62 58 223 55.75 Chetna Jain 90 85 95 91 361 90.25 Dinesh Kumar 40 52 58 65 215 53.75 --------------------------------------------------------------------------- SUBJECT-WISE COLUMN STATISTICS: ------------------------------------------- Subject Average Marks Status ------------------------------------------- Maths 67.50 GOOD Physics 65.75 GOOD Python 76.75 GOOD English 75.50 GOOD ---------- Class Topper : Chetna Jain (Total: 361/400) Low Avg Subjects : None
===========================================================================
```

9. Conclusion

The Student Marks Matrix Analysis program effectively utilizes nested 2D lists to derive both student-level (row-wise) and subject-level (column-wise) statistical insights, enabling automated identification of class toppers and low-performing subjects.